Proyecto Fashion Peaks

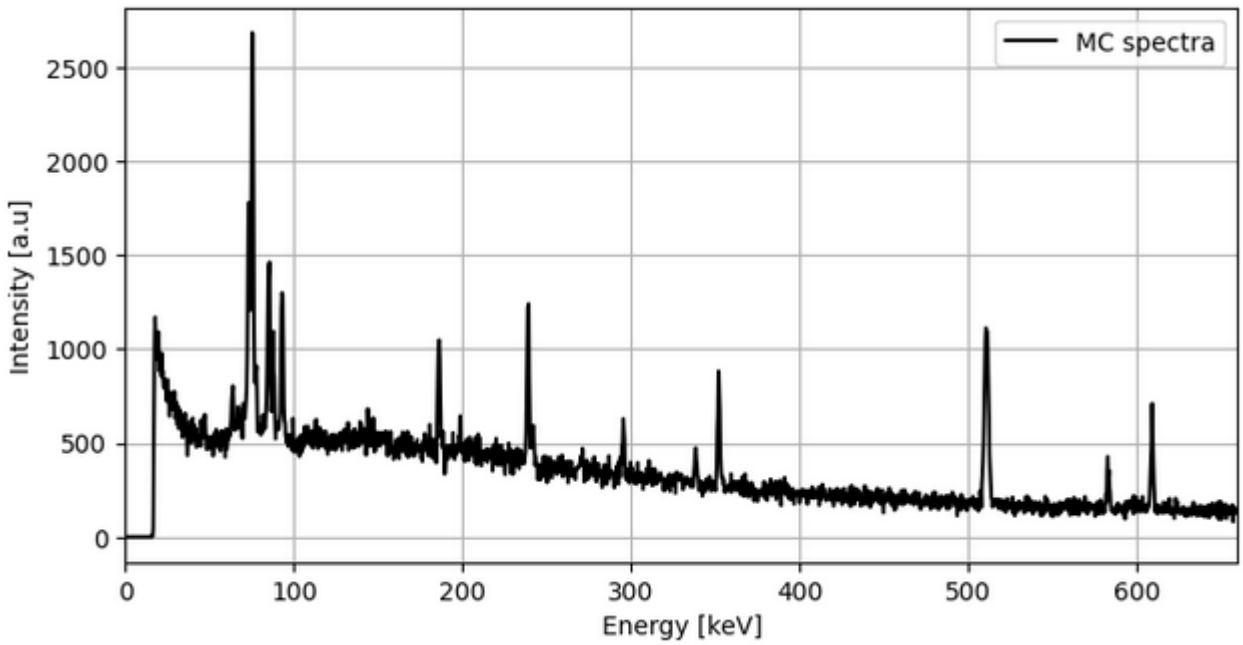
El nombre del proyecto **Fashion Peaks** surge como un juego de palabras inspirado en el trabajo con redes convolucionales (CNN) y el conjunto de datos **Fashion MNIST** [https://www.tensorflow.org/datasets/catalog/fashion_mnist].

La propuesta es determinar el área de picos producidos en espectros de rayos gamma por decaimiento radiactivo del Cesio 137, el cual es un isótopo radiactivo antropogénico.

A partir de sedimentos lacustres se miden espectros gamma naturales, en los cuales se observan picos característicos del Cs-137. Estos picos suelen ser pequeños y difíciles de procesar manualmente, especialmente cuando se dispone de grandes volúmenes de espectros provenientes de numerosas muestras.

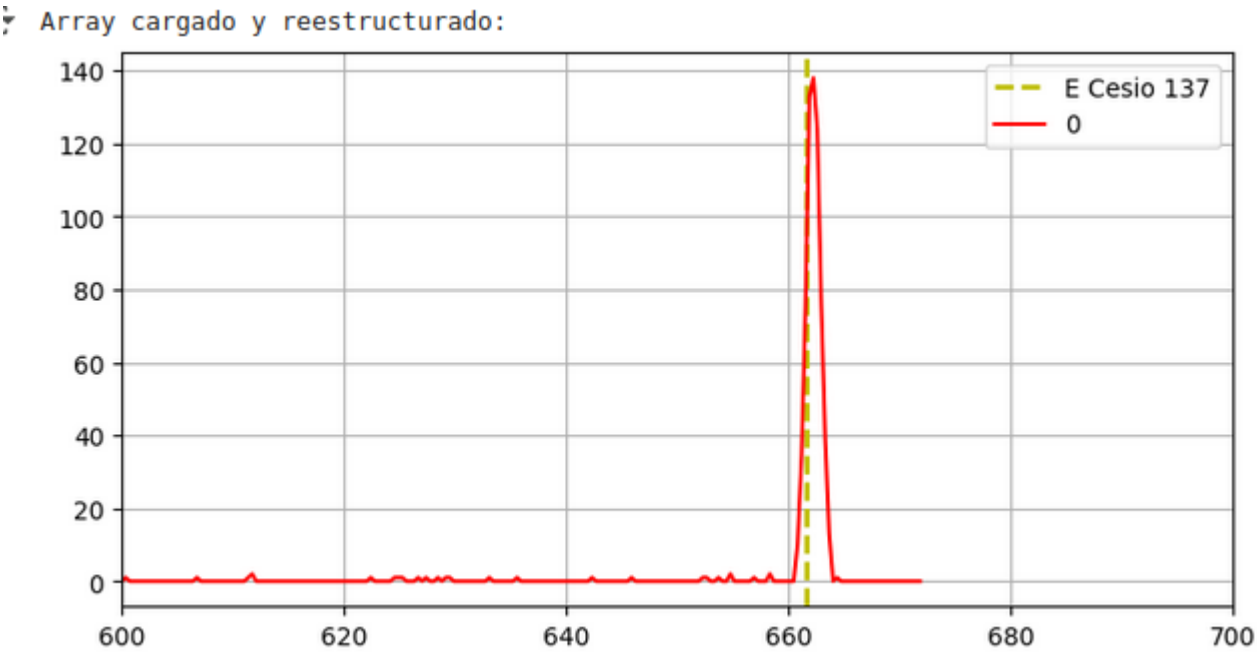
Los espectros gamma son medidos mediante un detector de alta energía de germanio hiperpuro, el cual está refrigerado a temperatura de nitrógeno líquido y acompañado por su cadena electrónica asociada.

En la siguiente figura se puede observar un espectro medido.



En la zona correspondiente a los 661 keV de energía es donde encontraremos los picos de Cs.

En la siguiente figura se presenta un pico de Cs simulado y amplificado para su observación.



El material adjunto en un archivo comprimido (zipeado), fue compactado a partir de una carpeta de Google Drive, en donde se corrieron 3 programas en notebook de Colab, los cuales son:

1. program_crea_de_corpus_v1.ipynb

Este notebook construye el corpus de entrenamiento de 2000 imágenes de picos de Cs. Las imagenes son de 32 pixeles de ancho por 64 pixeles de alto. Debajo se pueden observar seis imágenes:



Además, construye automáticamente los correspondientes labels, que se guardan en un archivo *CSV* llamado `labels.csv`.

Las imagenes están denominadas segun este nombre: spectro_xxxxx.png, en donde xxxxx va desde 00000 hasta 02000. Todo el corpus es guardado en la subcarpeta llamada `imagenes\_entrenamiento`, la carpeta raíz en este caso se llama `Fashion\_Peaks`.

Para ejecutar este código hay que vincular una cuenta de Google Drive al Google Colab y luego vincular con el path donde se encuentra el código y el corpus.

En este caso para hacerlo usamos:

```
from google.colab import drive
drive.mount('/content/gdrive')
```

Lo cual conecta el Google Colab con nuestra cuenta de Google Drive y luego necesitamo hacer:

```
path = '/content/gdrive/MyDrive/Colab Notebooks/Fashion_Peaks'
os.chdir(path)
```

aquí pondemos los nombres de nuestras carpetas del proyecto.

Recordemos que dentro de esta carpeta tenemos la sub carpeta **imagenes_entrenamiento** en donde hemos guardado el corpus para el entrenamiento.

Aclaración del tamaño del Corpus, actualmente tiene 2000 imágenes las cuales son clasificadas en 41 clases. He probado duplicar el tamaño y la CNN no aprendió nada, no se cual es el motivo, pero al utilizar de nuevo 2000 imágenes, se obtienen resultados alentadores.

Este código utiliza dos archivos de datos llamados: a. fileBase_2000x1900.dat b. fileCesio_2000x1900.dat

Cada archivo almacena 2000 espectros. En el caso del archivo `fileBase\_2000x1900.dat` son 2000 archivos de lo que se dice radiación de fondo, es medición que se realiza sin muestras y los rayos gamma medidos son radiación natural, cosmica, de los materiales de construcción del edificio, etc. Estos espectros fueron generados utilizando un método rápido de Monte Carlo (fast Monte Carlo).

El número 1900 corresponde a la cantidad de canales o bins en los que se ha clasificado la energía, ello significa que cada espectro tiene 1900 datos y en total el archivo contiene un total de 2000 × 1900 datos.

Y el archivo llamado `fileCesio\_2000x1900.dat` contiene 2000 espectros puros de Cesio, esto es sin radiación de fondo, simulados originalmente por el método convencional de Monte Carlo y código Penelope, simulando las condiciones de medición, esto es detector, blindaje, simulando sus formas y materiales. Y luego se crearon 2000 espectros mediante fast Monte Carlo. Estos 2000 espectros tienen distinto tamaño de área, para representar todo el barrido de las áreas de los picos esperados en las mediciones.

El corpus es producido por la selección aleatoria de un espectro del primer archivo y uno del segundo, sumados ambos espectros, luego son graficados y se selecciona la región de interés

en energía y la imagen generada se guarda con el nombre spectro_xxxxx.png, donde `xxxxx` va desde 00000 hasta 02000.

1.2 Labels

Los labels de cada imagen se corresponde con el área encerrado por los picos. Si bien se observa en el gráfico una pequeña región, pero en la realidad el pico se extiende hacia la izquierda hasta el cero. Aquí se muestra un espectro completo, en escala logaritmica, para que se pueda ver la cola que tiene hacia la izquierda, que si bien es muy baja relativa al pico, pero hay que considerar que esa cola está distribuida a todo lo largo, por lo que su área no es despreciable. Una vez creada el espectro, se mide el área correspondiente del especro de cesio, se multiplica por 1000, se clasifica en intervalos de 500 unidades, y se le asigna el centro de clase correspondiente. Es decir: si el área tiene entre 0 y 500 unidades, se le asigna la clase 250. Si tiene más de 500 y menos de 1000 se le asigna la clase de 750, y así sucesivamente. Esos labels están guardados en el archivo "imagenes_entrenamiento/labels.csv"

2. entrenamiento_01.ipynb

Este notebook tiene el código de entrenamiento de una red tipo CNN, cuyo nombre es **entrenamiento_01.ipynb**.

Este notebook se lo hizo para probar y mostrar como leer el corpus y que una CNN puede entrenarse con ellos.

se entrena una CNN durante 60 épocas utilizando las 2000 imágenes, se analiza el resultado de "loss" y "accuracy" y se guardan los pesos de la CNN entrenada, con el nombre "modelo_entrenado.pth".

La CNN se entrena para clasificar en 41 clases, por lo que hay transformación de los labels originales en 41 clases, del 0 al 40.

Actualmente se utiliza una CNN de **Pytorch** con la siguiente arquitectura:

class SpectraCNN(nn.Module): def **init**(self, n_classes=41): super(SpectraCNN, self).**init**()

```
        self.conv1 = nn.Conv2d(1, 16, kernel_size=3, padding=1)  # 1 canal → 16
        self.pool1 = nn.MaxPool2d(2, 2)  # reduce tamaño a la mitad

        self.conv2 = nn.Conv2d(16, 32, kernel_size=3, padding=1)
        self.pool2 = nn.MaxPool2d(2, 2)

        self.conv3 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
        self.pool3 = nn.AdaptiveAvgPool2d((1, 1))  # produce [64, 1, 1]

        self.fc1 = nn.Linear(64, 128)
        self.dropout = nn.Dropout(0.4)
        self.fc2 = nn.Linear(128, n_classes)  # salida final

    def forward(self, x):
        x = F.relu(self.conv1(x))  # [B,16,H,W]
        x = self.pool1(x)

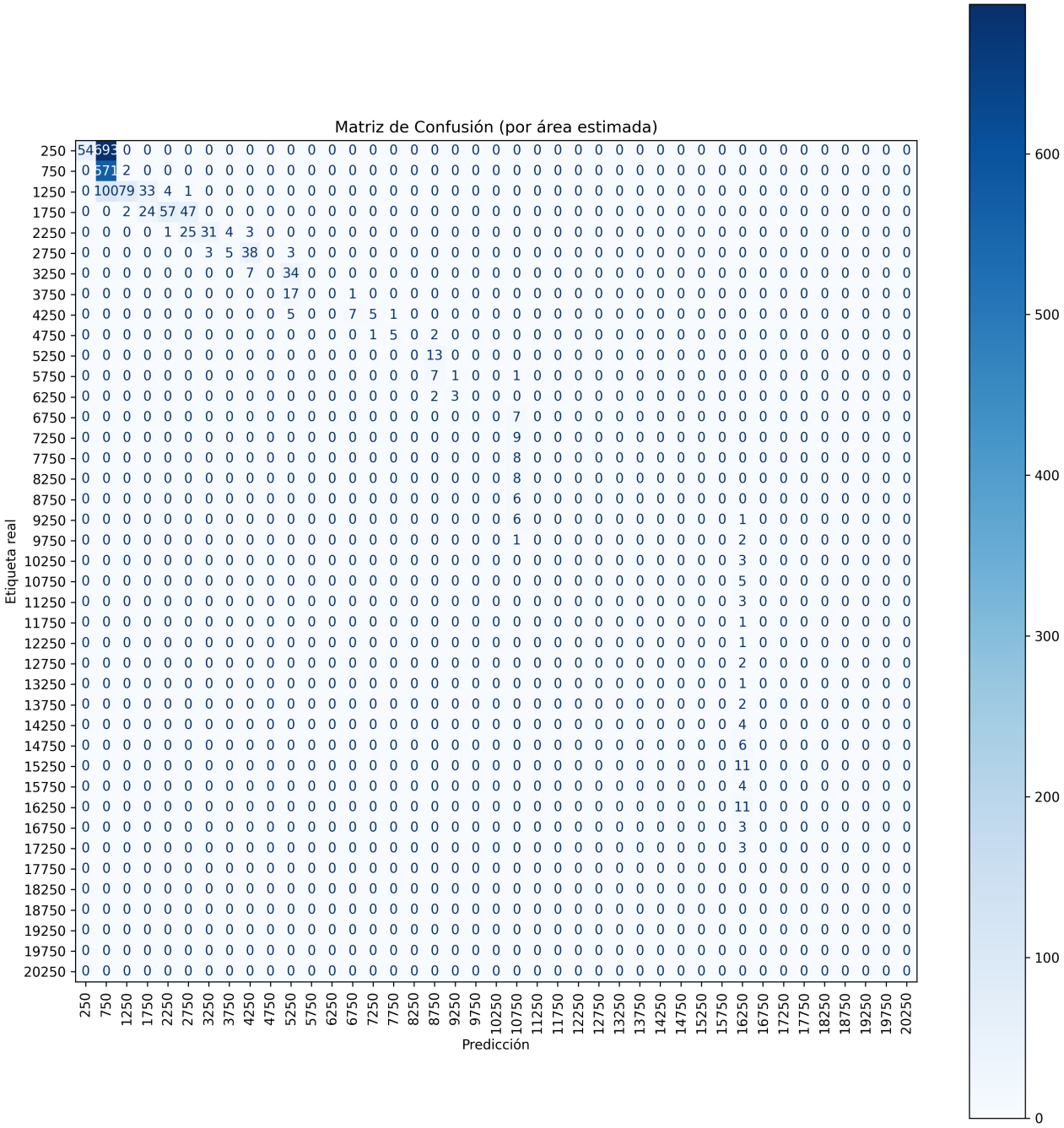
        x = F.relu(self.conv2(x))  # [B,32,H/2,W/2]
        x = self.pool2(x)

        x = F.relu(self.conv3(x))  # [B,64,H/4,W/4]
        x = self.pool3(x)          # [B,64,1,1]
        x = x.view(x.size(0), -1)  # flatten a [B,64]

        x = F.relu(self.fc1(x))
        x = self.dropout(x)
        x = self.fc2(x)
        return x
```

3. Leer_modelo_y_predecir_v0.ipynb

En este, tercer notebook, se leen los pesos de la CNN entrenada y se hacen algunas métricas para analizar el entrenamiento. Al final se crea la matriz de confusión se puede observar aquí:



De ella se desprende que la región de las áreas que van desde el valor de 9250 al 17250, la CNN tiende a clasificar erróneamente estas áreas dentro de la clase 16250. Por lo que se deberá trabajar para mejorar esta parte de clasificación.